where θ_b is the the parameters of pre-trained model that the reference and policy models are finetuned on, and $\tau_r, \tau^{(t)}$ are delta parameters $\tau_r = \theta_r - \theta_b$, $\tau^{(t)} = \theta^{(t)} - \theta_b$.

However, we empirically find directly optimizing Eq. 1 is unstable and hard to converge, and Eq. 1 requires an additional cache for pre-trained model parameters. Therefore, we apply a relaxation on Eq. 1 by moving the $\tau^{(t)}$ out of the merging and only focus on $\Delta\theta^{(t)}$, meaning we relax the merging for the past but only keep it toward the future update:

$$\theta^{(t+1)} \sim \theta_b + \tau^{(t)} + \mathcal{F}(\Delta\theta^{(t)}) \oplus \mathcal{F}(\tau_r) = \theta^{(t)} + \mathcal{F}(\Delta\theta^{(t)}) \oplus \mathcal{F}(\tau_r). \tag{2}$$

The delta parameter $\tau^{(t)}$ is already sparsified and consolidated with the reference in the previous update, so this relaxation still aligns well with our motivation for applying offline merging in optimization steps. Another essential benefit of this relaxation is avoiding caching additional parameters of θ_b, enhancing memory efficiency. With this relaxation, we show that the online merging at each optimization step can be approximated by consolidation between gradient-based delta weights and the delta parameters of the reference model.

4.2 Implementations

Note that our optimizer framework is highly flexible and compatible with exiting model merging methods. In this work, we develop two online merging optimizers, OnDARE and OnTIES, based on widely-used model merging methods, DARE (Yu et al., 2024) and TIES (Yadav et al., 2023).

OnDARE Optimizer. DARE employs a random sparsification method and a linear combination as the consensus method. So, the OnDARE optimizer following Eq. 2 is displayed in Eq. 3:

$$\theta_m^{(t)} = \theta_m^{(t-1)} + (1-\alpha)\mathcal{F}_R(\Delta\theta) + \alpha\mathcal{F}_R(\tau_r), \tag{3}$$

where α is a hyper-parameter controlling the weight of delta parameters of the reference model. Larger α introduces stronger regularization on the training. $F_R(\cdot)$ shown in Eq. 4 is a random sparsification operator based on the Bernoulli distribution with a fixed reserving probability p:

$$\mathcal{F}_R(x)_i = \begin{cases} x_i, & \text{Bernoulli}(p) = 1 \\ 0, & \text{Bernoulli}(p) = 0 \end{cases} \tag{4}$$

OnTIES Optimizer. TIES uses a top-k percentage sparsification and a sign-based consensus method. Specifically, it reserves the top-p percentage of parameters regarding absolute values from each merging candidate. It calculates the element-wise majority signs based on the sign and norms, drops parameters with different signs with the majority, and weighted sum up the rest. Similarly, the formulation of the OnTIES optimizer is shown in Eq. 5:

$$\theta_m^{(t)} = \theta_m^{(t-1)} + (1-\alpha)\mathcal{F}_R(\Delta\theta) \oplus_{sign} \alpha\mathcal{F}_R(\tau_r), \tag{5}$$

where $\mathcal{F}_R(\cdot)$ is a top-p percentage sparsification detailed in Eq. 6. The consensus method $\oplus_{sign}$ used in the OnTIES optimizer is a sign-based consensus among sparsified delta parameters as shown in Eq. 7, where $\mathbb{I}$ is the indicator function:

$$\mathcal{F}_R(x)_i = \begin{cases} x_i, & |x_i| \in \text{top-p}(|x|) \\ 0, & o.w. \end{cases} \tag{6}$$

$$a \oplus_{sign} b = \begin{cases} a + b, & \text{sign}(a) = \text{sign}(b) \\ a\mathbb{I}(|a| \geq |b|) + b\mathbb{I}(|b| \geq |a|), & \text{sign}(a) \neq \text{sign}(b) \end{cases} \tag{7}$$

The pseudo-code examples for these variants of online merging optimizers are shown in Alg. 1. It is worth noticing that we neglect the rescaling of sparsified delta parameters, which are proven to be essential in the offline merging (Yu et al., 2024; Yadav et al., 2023), as we find rescaling harms the numeric stabilities in multi-step optimization.

5 Experiments

5.1 Experimental Setup

Dataset. We conduct our experiment on the widely-used preference dataset ULTRAFEEDBACK. To be more specific, we use the binarized version of ULTRAFEEDBACK from Tunstall et al. (2023)[3], which is a reannotated and clean dataset for preference learning, compared with the original release (Cui et al., 2023). The training and evaluation splits of ULTRAFEEDBACK contain about 61K and 2K preference pairs, respectively, ranked by GPT-4, cleaned with manual efforts, and decontaminated with popular benchmarks such as TruthfulQA. The prompts in ULTRAFEEDBACK are large-scale, fine-grained, and diverse from multiple sources.

Training. We mainly explore our online merging optimizers in direct preference optimization (DPO, Rafailov et al. (2023)) on the ULTRAFEEDBACK dataset, as it is widely applied in nowadays large-scale LLM alignment due to its lower cost of training compared with Proximal Policy Optimization (PPO, (Schulman et al., 2017)). The general DPO includes sampling and annotating the responses from the policy model. In this work, we use an off-policy setting of DPO that directly trained our policy models on ULTRAFEEDBACK dataset, which has also proved effective in enhancing the helpfulness and harmlessness. The training loss of DPO is shown in Eq. 8, where β is the hyper-parameter controlling the strength of KL penalty between policy and reference models, π_θ and π_{ref} are policy and reference models, y_w and y_l are chosen and rejected samples in the preference pairs, respectively:

$$\mathcal{L}(\pi_\theta, \pi_{\text{ref}}) = -\mathbb{E}_{x,y_w,y_l\sim D}[\log\sigma(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)})]. \tag{8}$$

Note that online merging optimizers are agnostic with the training loss and optimizers, so they can be easily applied to PPO, other variants of DPO (Azar et al., 2023; Hong et al., 2024; Ethayarajh et al., 2024), and other online preference learning methods (Wu et al., 2024; Guo et al., 2024).

Evaluation. Assessing aligned large language models is a challenging task. We follow the straightforward principle that comprehensive evaluation yields trustworthy assessment. Therefore, our evaluation includes 12 public benchmarks in 7 categories evaluating the comprehensive abilities of aligned LLMs: **Mathematics**: (1) GSM8K (Cobbe et al., 2021) (2) Math401 (Yuan et al., 2023b) (3) Math23K (Wang et al., 2017); **Coding**: (1) HumanEval (Chen et al., 2021) (2) MBPP (Austin et al., 2021) (3) DS1000 (Lai et al., 2022) (4) CodeApex (Fu et al., 2024); **Instruction-following (IF)**: (1) IFEval (Zhou et al., 2023); **Reading Comprehension (RC)**: (1) COQA (Reddy et al., 2019) (2) DROP (Dua et al., 2019); **Knowledge** (1) MMLU (Hendrycks et al., 2021); **Agent** (1) NousResearch; **Code-switching**; We report the details of benchmarks in Appx. §B and inference configurations in Appx. §C. We use average scores within a category as the final category score and an average score among all benchmarks in all categories as the overall assessment. We also involve MT-Bench [4] and AlpacaEval 2.0 (Dubois et al., 2023, 2024) with length-controlled scoring, two leading and popular benchmarks that assess LLMs' alignment with human preferences using GPT-4-based evaluators.

Baselines. A naive baseline of our methods is the vanilla AdamW. We further consider offline merging methods, such as linear merging, DARE Yu et al. (2024), and TIES (Yadav et al., 2023) as our strong baselines as Lin et al. (2024) shows simple merging can mitigate alignment tax. As alignment tax is related to forgetting, we also involve traditional regularization methods, such as the KL penalty (Ouyang et al., 2022), EMA (Hunter, 1986; Noukhovitch et al., 2023), and ChildTuning (Xu et al., 2021) as our baselines. Specifically, the DPO algorithm adjusts the KL penalty with the hyper-parameter β. We also include LoRA (Hu et al., 2021) as one of our baselines, as parameter-efficient methods apply regularization regarding the weight space in the training (Biderman et al., 2024). Implementations of baselines are detailed in Appx. §C.

Configurations. We experiment with three LLM sizes, Qwen1.5-1.8B, Qwen1.5-7B, and LLaMa-3-8B series. Specifically, we use Qwen-1.8B-Base, Qwen-7B-Base, and LLaMa-3-8B as the base models for online optimizers. And we conduct direct preference optimization on ULTRAFEEDBACK on Qwen1.5-1.8B-SFT, Qwen1.5-7B-SFT, and LLaMa-3-8B-it as reference models. Two Qwen1.5 supervised finetuned models were trained on multi-lingual instruction data but no overlap with

[3] https://huggingface.co/datasets/HuggingFaceH4/ultrafeedback_binarized
[4] https://huggingface.co/spaces/lmsys/mt-bench

Table 1: Main results of baselines and online merging optimizers on ULTRAFEEDBACK trained from various backbone LLMs. In merging and dropping settings, we experiment with two variants, OnDARE and OnTIES. I.F., R.C., and Know. are short for instruction-following, reading comprehension, and knowledge. The Avg. columns are average scores except for MT-Bench and AlpacaEval 2.0. Benchmarks in each category are detailed in §5.1. We report the improvement against the vanilla AdamW (marked in gray). Higher scores are marked in darker green, while lower in darker red. The best and second-best results are marked in **bold** and underline, respectively.

	Method	Math	Code	I.F.	R.C.	Know.	Agent	CodeSwitch	Benchmark Avg.	MT-Bench	AlpacaEval 2.0 (LC)
Qwen1.5-1.8B-Chat	Reference	37.3	7.8	23.1	52.2	43.1	70.8	58.6	41.8	4.37	3.99
	AdamW	**37.8**	8.4	23.1	50.8	43.2	67.5	61.9	41.8	4.59	4.79
	Regularization										
	KL Penalty	-0.3	0.0	0.0	+0.5	+0.3	+1.2	+0.8	+0.4	0.00	+0.02
	EMA	-2.0	-0.5	-0.8	-1.4	+0.1	+3.0	+1.6	0.0	+0.09	-0.10
	ChildTuning	-0.9	+0.1	-0.2	+1.9	+0.7	0.0	+0.4	+0.3	+0.03	-0.04
	LoRA	-1.3	**+0.2**	+0.8	+1.6	+0.3	+0.3	-2.5	-0.1	-0.07	-0.23
	Offline Merging										
	Linear	-1.7	0.0	+0.8	-0.3	-0.2	+3.0	0.0	+0.2	-0.03	-0.96
	DARE	-2.4	-0.3	+0.2	-1.2	-0.1	**+5.6**	-0.8	+0.2	+0.09	-0.72
	TIES	-1.6	-0.2	+0.8	+0.2	-0.6	+1.5	-0.8	-0.1	-0.10	-0.41
	Online Merging (Ours)										
	OnTIES	-2.0	-0.6	+1.0	-0.5	+0.2	+1.5	**+2.0**	+0.3	+0.15	**+0.12**
	OnDARE	-1.3	-0.6	**+1.1**	**+4.9**	**+0.8**	-1.7	+0.4	**+0.5**	**+0.24**	+0.05
Qwen1.5-7B-Chat	Reference	65.5	23.0	34.3	69.2	59.1	74.6	75.0	57.3	6.97	10.71
	AdamW	69.3	22.1	38.0	71.9	59.4	76.3	71.7	58.4	7.18	11.66
	Regularization										
	KL Penalty	-1.9	+0.9	-2.2	-2.0	+0.1	+3.2	+2.9	+0.1	-0.23	+0.08
	EMA	-2.0	+0.5	-1.9	-3.0	-0.4	+1.5	+3.7	-0.2	-0.32	**+1.24**
	ChildTuning	-2.8	-0.47	+1.0	-2.1	-1.1	-0.6	+2.6	+2.5	-0.1	-0.11
	LoRA	-1.9	+0.4	-2.4	-2.1	-0.4	+2.4	**+5.3**	+0.2	-0.14	-0.33
	Offline Merging										
	Linear	-1.2	**+1.9**	-2.3	-2.1	-0.2	+1.5	+0.8	-0.2	-0.28	-1.65
	DARE	-1.8	+1.2	-2.1	-1.7	-0.5	+0.9	**+5.3**	+0.2	-0.13	-1.59
	TIES	-2.2	+1.7	-0.2	-1.4	-0.1	**+3.8**	+2.5	+0.6	-0.21	-1.35
	Online Merging (Ours)										
	OnTIES	-2.2	+0.1	+0.5	+0.3	**+1.0**	+2.4	+1.3	+0.5	-0.08	+0.91
	OnDARE	+0.1	+0.8	**+2.3**	+0.7	+0.3	-0.9	+4.1	**+1.1**	**+0.12**	+0.28
LLaMa-3-8B-Instruct	Reference	67.8	39.5	68.3	81.2	65.7	39.5	41.8	57.7	7.97	22.92
	AdamW	68.3	39.3	68.4	81.7	65.7	40.9	41.4	58.0	8.04	24.10
	Regularization										
	KL Penalty	**+1.9**	-0.8	-0.6	-0.4	**+0.1**	+1.2	+5.7	+1.0	+0.08	-1.02
	EMA	-2.9	-0.1	**+1.0**	-0.1	-0.1	+1.8	+2.5	+0.3	+0.18	+0.34
	ChildTuning	-1.5	-1.3	+0.2	-0.7	-0.7	-2.0	**+7.8**	+0.2	+0.14	+1.04
	LoRA	+0.1	-0.6	-0.2	-0.4	**+0.1**	**+4.1**	+1.6	+0.6	+0.07	-0.16
	Offline Merging										
	Linear	-1.1	-0.2	+0.3	-1.2	-0.5	-0.8	-4.5	-1.2	+0.11	+0.02
	DARE	-1.0	-0.3	-0.1	-0.2	-0.4	+1.5	+4.5	+0.5	+0.14	+0.18
	TIES	-1.2	**+0.3**	-0.3	-1.5	-0.3	+1.8	+1.2	0.0	+0.01	+0.21
	Online Merging (Ours)										
	OnTIES	-0.1	-0.7	0.0	-0.8	-0.1	+2.7	-0.4	0.0	+0.16	+0.84
	OnDARE	+0.1	-1.3	+0.8	**+0.5**	-0.2	+3.0	+6.6	**+1.3**	**+0.19**	**+1.57**

the ULTRAFEEDBACK dataset. More configuration about hyper-parameter searching, statistical significance, and experiment resources are detailed in Appx. §C

5.2 Main Results

We present our main results in Tab. 4, which showcases the performance of baseline methods and our proposed online merging optimizers on ULTRAFEEDBACK, trained from Qwen1.5-1.8B-Chat, Qwen1.5-7B-Chat, and LLaMa-3-8B-Chat. Overall, regularization and offline model merging methods do not significantly improve the average performance of RLHF models on the benchmarks compared with vanilla AdamW in most settings, and lead to a decrease in preference scores on MT-Bench and AlpacaEval 2.0. This indicates that simply relying on techniques such as dropout for gradients (ChildTuning), suppressing changes in model gradient updates (EMA), or making one-time adjustments to RLHF model parameters based on SFT models (Merging) are not effective solutions for the alignment bonus-tax trade-off. The regularization baselines particularly work well for LLama-3-8B-It, as all regularization methods achieves consistent improvements on the average benchmark scores, as well as MT-Bench and AlpacaEval scores. In contrast, our proposed Online Merging Optimizer, particularly the OnDARE variant, achieved the most significant improvements on all the test sets. OnDARE achieves the highest improvements on the benchmark average score and consistently enhances MT-Bench and AlpacaEval 2.0 on all three backbone LLMs, significantly surpasses other baselines, especially on the LLaMa-3-8B-Instruct experiments, where it achieves 1.3, 0.19, and 1.57 improvements on the benchmark, MT-Bench, and AlpacaEval, respectively. Though OnTIES and OnDARE show effectiveness in boosting rewards and mitigating tax, OnDARE is

slightly better than OnTIES in terms of the average benchmark scores in most cases, while OnTIES sometimes has higher LC win rates on AlpacaEval 2.0. Detailed scores of each benchmark are reported in Tab. 5, Tab. 6, Tab. 7.

5.3 Hyper-parameter Effects

This section analyzes how the two main hyper-parameters, parameter reserve rate p and merging weight α, affect the overall performance of online merging optimizers.

Parameter Reserve Rate is the reserve rate of parameters during online merging, which is p in Eq. 4 and Eq. 6. We explore reserve rates ranging from 1 to $1e^{-5}$ on Qwen1.5-1.8B-Chat to maximize the search space within our limited computational resources. As shown in Fig. 2, online merging optimizers remain robust even at low parameter reserve rates down to $5e^{-4}$. This indicates that discarding 99.95% of gradient-based parameter modifications at each RLHF step still results in stable training. OnTIES is more sensitive to extremely low parameter reserve rates compared to OnDARE. This sensitivity arises because OnDARE employs an unbiased random sparsification method, whereas the top-k sparsification used by OnTIES introduces significant bias during training.

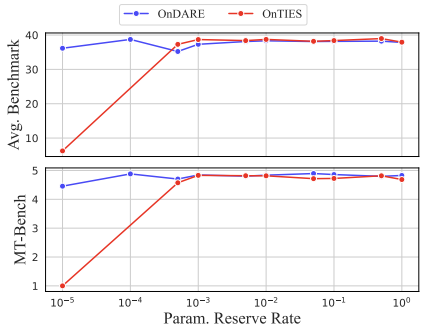

Figure 2: Analysis of Parameter Reserve Rates

Table 2: Analysis of the merging weight α with the OnDARE optimizer

α	Benchmarks(avg.)	MT-Bench
1e-4	7.4	1.00
5e-5	37.2	3.13
1e-5	40.1	4.37
5e-6	**41.8**	4.66
1e-6	41.6	4.76
5e-7	40.1	4.79
1e-7	39.2	**4.85**

Merging Weight is the aggregated weight for gradients and delta parameters of the reference (SFT) model. Larger merging weights introduce stronger regularization in the online merging optimizer. We experiment various merging weights α from 10^{-4} to 10^{-7}, and report the results at Tab. 2. As the merging weight increases, the MT-Bench scores rise due to fewer regularizations being added during training, while the average benchmark scores initially increase but then decrease, peaking at $\alpha = 5e^{-7}$. Similar to the exponential coefficient in the EMA training (Hunter, 1986), large α leads to unstable training. So, we suggest beginning the hyper-parameter searching of α from a minor number such as 10^{-7}. It is worth noticing that a special case $\alpha = 0$ makes OnDARE equivalent to gradient dropout regularization methods, such as ChildTuning (Xu et al., 2021) in our baselines.

5.4 The Impact of RLHF Algorithms

In §5.1, we have validated the effectiveness of our Online Merging Optimizers in compatibility with DPO. In this section, we further study its application on other RLHF algorithms. In this section, we further explore their application to other RLHF algorithms. Specifically, we implement OnDARE and OnTIES with IPO (Azar et al., 2023) and KTO (Ethayarajh et al., 2024). We train Qwen1.5-7B-Chat under these settings on the ULTRAFEEDBACK datasets and present the results in Tab. 3. The Online Merging Optimizers, OnDARE, and OnTIES outperform AdamW across all algorithms on MT-Bench, except for the average benchmark score with the IPO algorithm. This demonstrates that their effectiveness extends to multiple RLHF algorithm variants.

Table 3: Analysis on different RLHF algorithms with online merging optimizers. We report the average scores of benchmark and MT-Bench scores as Tab. 4. B.M. is short for Benchmark. Best scores are marked in **bold**.

Method	Optimizer	B.M.(avg.)	MT-Bench
IPO	AdamW	**58.1**	7.06
	OnDARE	57.0	7.09
	OnTIES	57.3	**7.34**
KTO	AdamW	58.3	7.18
	OnDARE	58.6	**7.46**
	OnTIES	**59.3**	7.42